

AppData

AppData · docs/latest · generated 2026-08-15

AppData is the platform's persisted, cross-window **key/value store**, modeled as a *kind of data provider* (`providerType: 'appdata'`). Its values are inlined into other providers' configs via `{{name.key}}` template substitution, and grids read it for dynamic cell-editor dropdowns and the historical-date (`asOfDate`) workflow.

It is **not** a stream: an AppData row has no start/stop/snapshot lifecycle and never registers with the provider factory. Think "named bags of settings that every window agrees on".

The layers

LAYER	SYMBOL	WHERE
Worker (authority)	<code>HubAppDataService</code>	<code>@wellsfargo-starui/data</code> — the single in-memory store per hub and the only IndexedDB writer
Main-thread mirror	<code>AppDataMirror</code>	<code>@wellsfargo-starui/data</code> — sync-read RPC client; never touches Dexie itself
Persistence	<code>AppDataConfigStore</code>	ConfigManager/Dexie rows
React	<code>useAppDataStore()</code> , <code>useAppData(name)</code>	<code>@wellsfargo-starui/react/data/runtime</code>
Grid engine	<code>AppDataLookup</code> (interface)	<code>@wellsfargo-starui/core</code> — narrow read/write adapter with no dependency on the data package

A write flows: `set()` → hub persists to IndexedDB → hub broadcasts the delta → **every** attached mirror (including the writer's) applies it → the returned promise resolves. Durable *and* consistent when it settles.

Visibility model — global, not scoped

There is no app/user/grid scope enum. Rows are scoped only by:

- **name** — the `{{name.key}}` namespace, and
- **ownership** — `isPublic: true` rows are owned by `'system'` (visible to everyone sharing the appId); otherwise the authoring user.

Listing is deliberately **platform-global**: rows authored under any app surface in `{{name.key}}` resolution anywhere.

Two row shapes coexist in storage (both flatten to the same `AppDataConfig`): legacy standalone rows (`componentType: 'appdata'`) and unified provider-editor rows (`componentType: 'data-provider'`, `componentSubType: 'appdata'`, whose `variables` map is unwrapped to scalars — `{{App1Data.userId}}` resolves the value, not the `AppDataVariable` wrapper).

Reading and writing

AppDataMirror (from `useAppDataStore().store`, or `platform.appData`):

MEMBER	SEMANTICS
<code>get(name, key)</code>	sync read; <code>undefined</code> for unknown name/key or pre-snapshot
<code>list()</code> / <code>isReady()</code> / <code>ready()</code>	row snapshot; readiness (initial snapshot arrived)
<code>set(name, key, value)</code>	creates the row if missing; resolves after persist + every-mirror apply
<code>publishNamedRow(name, values)</code>	write every key of a named row in ONE hub round-trip (merges by name)
<code>upsertConfig(cfg)</code> / <code>remove(configId)</code>	whole-row upsert / delete
<code>subscribe(fn)</code>	any-change notification (after the local mirror updated)

React:

```
import { useAppData } from '@wellsfargo-starui/react/data/runtime';

const { values, loaded, get, set } = useAppData('positions');
await set('asOfDate', '2026-04-30'); // durable + broadcast when resolved
```

`useAppDataStore()` returns `{ store, version, loaded }` — `version` bumps on every mutation and is the dependency to re-run template resolution. `useResolvedCfg(cfg)` resolves a provider config's templates and swaps identity **only when a key the config actually references changes**.

Grids consume AppData through the core `AppDataLookup` interface (`get`, optional `listProviders` / `keysOf` / `subscribe` / `set`) plumbed via `GridPlatformOptions.appData` → `resources.appData()`. Consumers must handle its absence — AppData-tied features simply don't render. The three in-grid consumers: cell-editor dropdown values via `valuesSource: '{{providerName.key}}'`, the column-settings picker, and hot-reload subscriptions.

Template resolution — `{{name.key}}`

Any string leaf in a provider config can reference AppData:

- `{{name.key}}` → `appData.get(name, key)`
- `{{name.a.b.c}}` → first segment is the row name, the rest indexes into a JSON value

- Tokens that don't resolve are **left in place verbatim** — a debugging affordance, never silent corruption. The SharedWorker additionally **fails closed**: `findUnresolvedAppDataTokens` / `assertAppDataResolved` reject a config with unresolved tokens before anything reaches the broker wire, and the STOMP transport refuses `{{` in `listenerTopic` / `requestMessage` / `requestBody`.

Common entries: `positions.asOfDate` (historical date picker), `positions.clientId` (account scope token), `auth.token` (bearer token shared across REST configs).

The historical-date workflow (`asOfDate`)

The concrete AppData use-case wired end-to-end (CSRM only):

1. The grid's **Custom Settings** panel enables it (`historicalDateAppDataEnabled` + provider name + key).
2. Picking a past toolbar date writes ISO `YYYY-MM-DD` to that AppData key.
3. The historical provider's config references it — e.g. `{{positions.asOfDate}}` in a topic or request body — and the container restarts the provider (`historicalDateAppDataRef: 'positions.asOfDate'`).
4. On restart, an overlay carrying `asOfDate` **wins** over possibly-stale AppData for historical-date keys, so toolbar reload is deterministic.

SSRM has **no historical-date counterpart yet** — `historicalDateAppDataRef` is deliberately absent from the SSRM container.

Bootstrap hooks

Apps seed AppData at platform boot through a hook registry (`AppDataBootstrapHookRegistry` from `@wellsfargo-starui/data`), declared in the platform bootstrap config (`appDataBootstrap.onHubReady` / `onUserChange`), with run policies:

POLICY	RUNS
<code>if-missing</code> (default)	only when a target row name has no values yet
<code>once-per-session</code>	guarded by a sessionStorage sentinel (<code>starui:appDataBootstrap:<appId>:<userId>:<hookId></code>)
<code>always</code>	every boot

Hook failures log-and-continue unless `strict: true`. Worked example: `apps/source/stomp-marketsgrid-minimal/src/platform/appDataBootstrap.ts` (seeds `SessionContext` with `userId`, entitlements, and `position-asofdate`). The full walkthrough is in `docs/guides/platform-bootstrap-config.md`.

Two naming traps

- **AppDataLookup** is two different types with one name: the function form `(providerName, key) => unknown` (template resolver, `@wellsfargo-starui/data`) and the object interface (grid engine, `@wellsfargo-starui/core`). Both are real; check which package you're importing from.
- **createAppDataServices** is not **AppData** — it is "create App DataServices", the SharedWorker bundle bootstrap.

Editing

The Data Provider Editor's **AppData** transport panel edits rows as key/value tables (`npm run app -- dataprovider-editor`); the Hub Inspector drawer (Alt+Shift+S in dev) shows the live rows the worker holds.